Agentic AI Bootcamp

Applications: Teaching & Research

Erkmen G. Aslim & **Emily Beam**

Department of Economics, University of Vermont

Session 2 · April 27, 2026 · Old Mill A500

thinkingwithagents.github.io

Where We Left Off & Today's Plan

From Session 1

- Agentic AI vs. chat AI
- Context window & planning
- `CLAUDE.md`, `PLAN.md`, `SKILL.md`
- Permissions & safety
- App 1 (live): website redesign
- App 2 (designed): **lecture-builder**

Today: Three Live Applications

1. **Lecture Builder** — create the skill we designed last time, then run it on real papers
2. **Research Brainstorm** — stress-test an audience question live
3. **Web Scraper** — novel data on screen in 10 minutes

Each app is end-to-end: prompt → skill → output. Prompts are on the slides — copy as we go.

Setup: Install Your Skill Toolkit

Anthropic skills (one-time)

1. **Add the marketplace:**

```
/plugin marketplace add anthropics/skills
```

2. **Install the document-skills bundle:**

```
/plugin install
  document-skills@anthropic-agent-skills
```

Gives you: `pdf`, `docx`, `pptx`, `xlsx`, `frontend-design`, `skill-creator`, `algorithmic-art`.

Third-party skills via `npx`

Full economist pack:

```
npx skills add thinkingwithagents/skills
```

One skill at a time:

```
npx skills add thinkingwithagents/skills
  --skill research-brainstorm
```

Includes: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

Setup: Install Your Skill Toolkit

Anthropic skills (one-time)

1. Add the marketplace:

```
/plugin marketplace add anthropics/skills
```

2. Install the document-skills bundle:

```
/plugin install
  document-skills@anthropic-agent-skills
```

Gives you: `pdf`, `docx`, `pptx`, `xlsx`, `frontend-design`, `skill-creator`, `algorithmic-art`.

Third-party skills via `npx`

Full economist pack:

```
npx skills add thinkingwithagents/skills
```

One skill at a time:

```
npx skills add thinkingwithagents/skills
  --skill research-brainstorm
```

Includes: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

! `/skill-creator` is **not** bundled with Claude Code by default — it ships inside `document-skills`. Install it before App 1.

Application 1

Lecture Builder Skill — From Papers to a Lecture

Where We Left Off — The Skill Spec

```
/skill-creator

Create a skill called "lecture-builder" that converts academic documents into lecture materials.  The skill should:

1.  Read documents in their entirety (PDF, DOCX, web pages) -- break reading into parts to avoid excessive token

usage.  Include a Python parsing script.

2.  Extract:  key arguments, empirical findings, methodological details, discussion points, potential exam questions.

3.  Generate two outputs:

a) Structured lecture notes (markdown) with section headers, key takeaways, discussion prompts

b) A polished slide deck (draw on \academic-beamer-deck or \pptx) with clean visuals, one idea per slide

4.  Adapt tone for undergraduate vs. graduate audiences (configurable).

Save SKILL.md + supporting files to .claude/skills/lecture-builder/.
```

Last week we wrote the spec. ***Today we actually build the skill — then run it.***

Building the Skill: `/skill-creator` Walkthrough

Building the Skill: `/skill-creator` Walkthrough

`/skill-creator` **interviews you**. Be ready to answer:
name, audience, inputs, outputs, edge cases, where to save.

Anatomy of `lecture-builder`

`.claude/skills/lecture-builder/`

- `SKILL.md` playbook
- `parse_document.py` PDF/DOCX reader
- `slide_template.tex` UVM Beamer
- `notes_template.md` notes scaffold

What `SKILL.md` enforces

- Chunked reading → token control
- Mandatory citations from source PDFs
- Undergrad vs. grad tone toggle
- Dual output: notes + slides
- Reuses UVM color palette

Anatomy of `lecture-builder`

`.claude/skills/lecture-builder/`

- `SKILL.md` playbook
- `parse_document.py` PDF/DOCX reader
- `slide_template.tex` UVM Beamer
- `notes_template.md` notes scaffold

What `SKILL.md` enforces

- Chunked reading → token control
- Mandatory citations from source PDFs
- Undergrad vs. grad tone toggle
- Dual output: notes + slides
- Reuses UVM color palette

Skills with Python scripts > pure-markdown skills when tokens matter. The script reads; the LLM synthesizes.

The Input Folder

`./inputs/`

- `gov_thinktank_ice_arrests.pdf` — policy report
- `ice_enforcement_report.pdf` — government brief
- `aslim_paper.pdf` — academic working paper

Why this mix?

- Heterogeneous sources: **policy** + **government** + **academic**
- Forces the skill to harmonize tone, terminology, and evidence quality
- Mirrors how you'd actually build a lecture

The Input Folder

`./inputs/`

- `gov_thinktank_ice_arrests.pdf` — policy report
- `ice_enforcement_report.pdf` — government brief
- `aslim_paper.pdf` — academic working paper

Why this mix?

- Heterogeneous sources: **policy** + **government** + **academic**
- Forces the skill to harmonize tone, terminology, and evidence quality
- Mirrors how you'd actually build a lecture

Topic: The labor and health consequences of ICE enforcement. Connects directly to ongoing research at UVM.

The Execution Prompt

```
/lecture-builder

I have a folder ./inputs/ with:  gov_thinktank_ice_arrests.pdf,
ice_enforcement_report.pdf, aslim_paper.pdf.

Build a 50-minute undergraduate lecture on the labor and health
consequences of ICE enforcement.  Audience:  junior/senior econ majors
who have taken intermediate micro and one applied econometrics class.

Outputs to ./outputs/:
  1) lecture_notes.md -- sectioned notes with 5 key takeaways,
     4 discussion prompts, 3 candidate exam questions.
  2) lecture_slides.tex -- Beamer deck (~20 slides, 16:9).  Use
     a clean academic palette -- default UVM (deep green / gold) or ask
     me.  In-text citations to all three source PDFs.

Be conservative with tokens:  parse each PDF in chunks via
parse_document.py, summarize per-section, then synthesize.
```

Audience spec + output spec + token discipline. Skills work better when the prompt is concrete.

Expected Output (Live Demo Running)

▶ Live demo running in the background — final files appear by the end of this section

lecture_notes.md

 # 1. The setting
Recent expansions of interior immigration enforcement...
 # 2. Identification strategy
287(g) rollout as a staggered DiD...
 # 3. Findings
Aslim et al. document...
 # Discussion prompts
1. Why might the effect on infant health attenuate over time?
 # Exam questions
1. Sketch the parallel-trends test...

synthesizes →

lecture_slides.tex

```
% 20 slides, 16:9
% UVM palette
\begin{frame}{Setting}
  ICE arrests timeline
\end{frame}
\begin{frame}{Methods}
  staggered DiD
\end{frame}
\begin{frame}{Results}
  main effect
\end{frame}
```

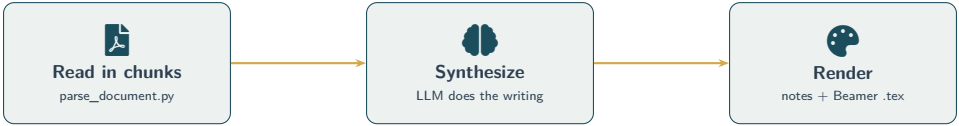
What Just Happened (Under the Hood)

Read in chunks

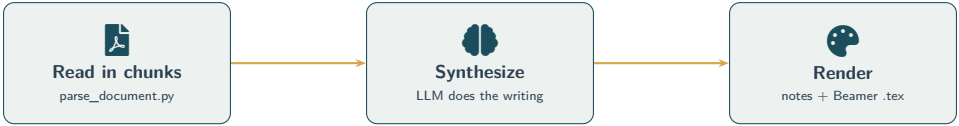
parse_document.py

What Just Happened (Under the Hood)

What Just Happened (Under the Hood)



What Just Happened (Under the Hood)



This skill is reusable. Swap the input folder, swap the topic, swap the audience — get a new lecture in minutes. Build once, deploy forever.

Application 2

`/research-brainstorm` — Stress-Test a Research Idea

What the Research Brief Contains

Brief sections

- **The question** — structured: unit, population, treatment, outcome, comparison, setting
- **Closest related work** — 3–6 cites with sharp delta
- **Contribution type** — setting / identification / reframing / measurement / mechanism / policy
- **The strongest objection** — one paragraph, unvarnished
- **Alternative framings** — 2–3 pivots
- **Feasibility** — data, identification, power, timeline, fatal risks
- **Next concrete step**

Saved to:

`research_briefs/YYYY-MM-DD-slug.md`

Markdown so it lives next to your other notes. Re-openable, editable, citeable in your own write-ups.

What the Research Brief Contains

Brief sections

- **The question** — structured: unit, population, treatment, outcome, comparison, setting
- **Closest related work** — 3–6 cites with sharp delta
- **Contribution type** — setting / identification / reframing / measurement / mechanism / policy
- **The strongest objection** — one paragraph, unvarnished
- **Alternative framings** — 2–3 pivots
- **Feasibility** — data, identification, power, timeline, fatal risks
- **Next concrete step**

Saved to:

`research_briefs/YYYY-MM-DD-slug.md`

Markdown so it lives next to your other notes. Re-openable, editable, citeable in your own write-ups.

The conversation is great.
The artifact is what you keep.

Installing /research-brainstorm

Full economist pack

```
npx skills add
  thinkingwithagents/skills
```

Installs the full pack in one go. Today we use: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

One skill at a time

```
npx skills add
  thinkingwithagents/skills
  --skill research-brainstorm
```

Mix-and-match per project. Lighter footprint when you only need one.

Installing /research-brainstorm

Full economist pack

```
npx skills add
  thinkingwithagents/skills
```

Installs the full pack in one go. Today we use: `research-brainstorm`, `find-data`, `academic-beamer-deck`.

One skill at a time

```
npx skills add
  thinkingwithagents/skills
  --skill research-brainstorm
```

Mix-and-match per project. Lighter footprint when you only need one.

Same `npx skills` mechanism works for any GitHub-hosted skill repo. **Build your own pack and share it with your coauthors.**

Live Demo: Hand Me a Question

Audience:

hand me a half-formed research question.

Live Demo: Hand Me a Question

Audience:
hand me a half-formed research question.

An effect
you suspect
but can't yet measure

A dataset
you have access to
but haven't used

A policy
you want
to evaluate

Live: invoke `/research-brainstorm`, ride through the phases, walk away with a brief.

Application 3

Web Scraper — From `/find-data` to a Live Visualization

`/find-data` Picks NBER Working Papers

`/find-data` in one line

We hand it our constraints (applied-micro audience, 24+ months, scrape-able, time-series + cross-section). It scans repositories and surfaces a source.

Today's pick: NBER working papers.

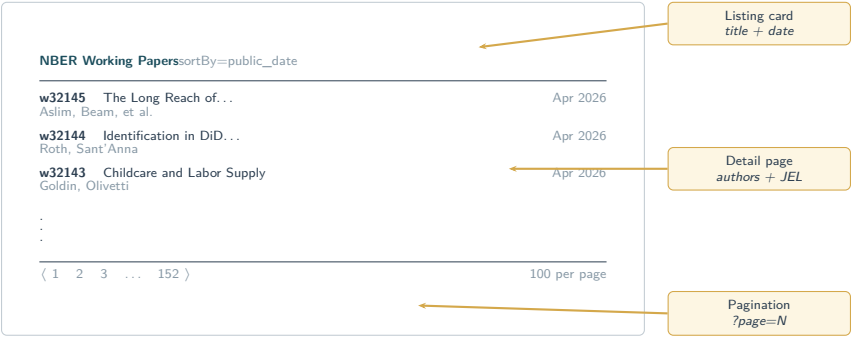
Why NBER: every economist cares; clean HTML; rich JEL + affiliation cross-section; ~1,500 papers / year.

URL:
`nber.org/papers?page=N&perPage=100&sortBy=public_date`

Extract: `nber_number, title, authors, public_date, primary_jel_code, affiliations.`

Goal: novel data on the screen in under 10 minutes. Skip the buildup — go straight to scraping.

Inspect Before You Prompt



View-source first. The agent writes better scrapers when you've already named the selectors.

The Scraping Prompt

```
Scrape NBER working papers for a teaching demo.  Build a self-contained

Python script in ./scrape_nber/ using uv:

1. Fetch the most recent ~1,000 papers from

   nber.org/papers?page=N&perPage=100&sortBy=public_date

   (paginate until 1,000 or cutoff 2024-01-01).

2. Extract:  nber_number, title, authors, public_date, primary

   JEL code, affiliations (from detail page if needed).

3. Save to ./scrape_nber/data/nber_papers.csv.

4. Generate three PNGs to ./scrape_nber/figures/:

   - papers_per_month.png   bars, last 24 months, UVM colors

   - top_jel_codes.png   top 15 primary JEL codes

   - top_institutions.png   top 10 affiliations

5. Print 5-line stdout summary.

Politeness:  real-browser User-Agent; sleep 0.5s between fetches;

cache HTML to ./scrape_nber/cache/ so re-runs are free.
```

What Claude Does While We Keep Talking

Fetching pages...

Parsing...

Plotting...

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What Claude Does While We Keep Talking

✔ Fetched 10 / 10 pages

Parsing 1,000 records. . .

Plotting. . .

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What Claude Does While We Keep Talking

✔ Fetched 10 / 10 pages

✔ Parsed 1,000 records

◌ **Plotting 3 figures. . .**

```
$ uv run scrape_nber/main.py

[INFO] fetching page 1/10 ...ok (98 papers)

[INFO] fetching page 2/10 ...ok (100 papers)

[INFO] ...

[INFO] parsing JEL codes ...

[INFO] saved data/nber_papers.csv

[INFO] writing figures/...

== Summary ==

n_papers:  1,000

date range:  2024-04 to 2026-04

top JEL: J (Labor)

top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What Claude Does While We Keep Talking

- ✔ Fetched 10 / 10 pages
- ✔ Parsed 1,000 records
- ✔ Wrote 3 PNGs

```
$ uv run scrape_nber/main.py
[INFO] fetching page 1/10 ...ok (98 papers)
[INFO] fetching page 2/10 ...ok (100 papers)
[INFO] ...
[INFO] parsing JEL codes ...
[INFO] saved data/nber_papers.csv
[INFO] writing figures/...

== Summary ==
n_papers:  1,000
date range:  2024-04 to 2026-04
top JEL: J (Labor)
top affil:  NBER, Harvard, MIT
```

The terminal is a TV. Let it run while you talk through what's coming.

What We Built Today

1. Built and ran a real teaching skill — **lecture-builder**

What We Built Today

1. Built and ran a real teaching skill — **lecture-builder**
2. Stress-tested a research idea with **evidence-grounded critique**

What We Built Today

1. Built and ran a real teaching skill — **lecture-builder**
2. Stress-tested a research idea with **evidence-grounded critique**
3. Went from “what data?” to “here’s a chart” in **under 10 minutes**

What We Built Today

1. Built and ran a real teaching skill — **lecture-builder**
2. Stress-tested a research idea with **evidence-grounded critique**
3. Went from “what data?” to “here’s a chart” in **under 10 minutes**

Three workflows. Three artifacts on disk. **All reusable.**

Thinking With Agents

Online sessions coming this summer

Workshops, deep-dives, and Q&A on agentic AI for academic research — built by economists, for economists and academics alike.

Stay tuned for dates and registration:

`thinkingwithagents.github.io`